

System Programming

Conditional Processing

Dongwann Kang

Assistant Professor
Department of Computer Science and Engineering

Boolean and Comparison Instructions

- The Intel instruction set contains the AND, OR, XOR, and NOT instructions, which directly implement boolean operations on binary bits

Operation	Description
AND	Boolean AND operation between a source operand and a destination operand.
OR	Boolean OR operation between a source operand and a destination operand.
XOR	Boolean exclusive-OR operation between a source operand and a destination operand.
NOT	Boolean NOT operation on a destination operand.
TEST	Implied boolean AND operation between a source and destination operand, setting the CPU flags appropriately.

- 1. The CPU Status Flags
 - Boolean instructions affect the Zero, Carry, Sign, Overflow, and Parity flags
 - The Zero flag is set when the result of an operation equals zero
 - The Carry flag is set when an operation generates a carry out of the highest bit of the destination operand
 - The Sign flag is a copy of the high bit of the destination operand
 - The Overflow flag is set when an instruction generates a result that is outside the signed range of the destination operand
 - The Parity flag is set when an instruction generates an even number of 1 bits in the low byte of the destination operand

Boolean and Comparison Instructions

- 2. AND Instruction

- The AND instruction performs a boolean (bitwise) AND operation between each pair of matching bits in two operands and places the result in the destination operand

`AND destination, source`

- The following operand combinations are permitted:

`AND reg, reg` `AND mem, reg`
`AND reg, mem` `AND mem, imm`
`AND reg, imm`

x	y	$x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

- The operands can be 8, 16, 32, or 64 bits, and they must be the same size
- Immediate operands can be no larger than 32 bits

- Bit masking

- The AND instruction lets you clear 1 or more bits in an operand without affecting other bits
- E.g.) A control byte is about to be copied from the AL register to a hardware device.
Further, we will assume that the device resets itself when bits 0 and 3 are cleared in the control byte.
Assuming that we want to reset the device without modifying any other bits in AL, we can write the following:

`and AL, 11110110b`

- Flags

- The AND instruction always clears the Overflow and Carry flags
- It modifies the Sign, Zero, and Parity flags according to the destination operand

Boolean and Comparison Instructions

- 2. AND Instruction

- Converting Characters to Upper case

- The AND instruction provides an easy way to translate a letter from lowercase to uppercase
 - If we compare the ASCII codes of capital A and lowercase a, it becomes clear that only bit 5 is different

0 1 **1** 0 0 0 0 1 = 61h ('a')
0 1 **0** 0 0 0 0 1 = 41h ('A')

- The rest of the alphabetic characters have the same relationship
 - If we AND any character with 11011111 binary, all bits are unchanged except for bit 5, which is cleared
- In the following example, all characters in an array are converted to uppercase

```
.data
array BYTE 50 DUP(?)
.code
    mov     ecx,LENGTHOF array
    mov     esi,OFFSET array
L1:  and     BYTE PTR [esi],11011111b    ; clear bit 5
    inc     esi
    loop    L1
```

Boolean and Comparison Instructions

- 3. OR Instruction

- The OR instruction performs a boolean OR operation between each pair of matching bits in two operands and places the result in the destination operand

`OR destination, source`

- The following operand combinations are permitted:

`OR reg, reg` `OR mem, reg`
`OR reg, mem` `OR mem, imm`
`OR reg, imm`

x	y	$x \vee y$
0	0	0
0	1	1
1	0	1
1	1	1

- The operands can be 8, 16, 32, or 64 bits, and they must be the same size
- For each matching bit in the two operands, the output bit is 1 when at least one of the input bits is 1

- The OR instruction is particularly useful when you need to set 1 or more bits in an operand without affecting any other bits
 - E.g.) Suppose that your computer is attached to a servo motor, which is activated by setting bit 2 in its control byte. Assuming that the AL register contains a control byte in which each bit contains some important information, the following code only sets the bit in position 2

`or AL, 00000100b`

- Flags

- The OR instruction always clears the Carry and Overflow flags
- It modifies the Sign, Zero, and Parity flags according to the destination operand

Boolean and Comparison Instructions

- 4. Bit-Mapped Sets
 - Some applications manipulate sets of items selected from a limited-sized universal set
 - E.g.) Employees within a company, environmental readings from a weather monitoring station
 - In such cases, binary bits can indicate set membership
 - Rather than holding pointers or references to objects in a container, an application can use a bit vector to map the bits in a binary number to an array of objects
 - An example
 - The following binary number uses bit positions numbered from 0 on the right to 31 on the left to indicate that array elements 0, 1, 2, and 31 are members of the set named SetX

`SetX = 10000000 00000000 00000000 00000111`

(The bytes have been separated to improve readability)

- We can easily check for set membership by ANDing a particular member's bit position with a 1

```
mov  eax,SetX
and  eax,10000b ; is element[4] a member of SetX?
```

- If the AND instruction in this example clears the Zero flag, we know that element [4] is a member of SetX

Boolean and Comparison Instructions

- 4. Bit-Mapped Sets

- Set Complement

- The complement of a set can be generated using the NOT instruction, which reverses all bits
 - Therefore, the complement of the SetX is generated in EAX using the following instructions:

```
mov  eax,SetX
not  eax      ; complement of SetX
```

- Set Intersection

- The AND instruction produces a bit vector that represents the intersection of two sets
 - The following code generates and stores the intersection of SetX and SetY in EAX:

```
mov  eax,SetX
and  eax,SetY
```

- This is how the intersection of SetX and SetY is produced:

	10000000000000000000000000000000111	(SetX)
AND	1000001010100000000011101100011	(SetY)

	1000000000000000000000000000000011	(intersection)

- It is hard to imagine any faster way to generate a set intersection
 - A larger domain would require more bits than could be held in a single register, making it necessary to use a loop to AND all of the bits together

Boolean and Comparison Instructions

- 4. Bit-Mapped Sets
 - Set Union
 - The OR instruction produces a bit map that represents the union of two sets
 - The following code generates the union of SetX and SetY in EAX

```
mov    eax, SetX
or     eax, SetY
```

- This is how the union of SetX and SetY is generated by the OR instruction:

	10000000000000000000000000000111	(SetX)
OR	10000010101000000000011101100011	(SetY)

	10000010101000000000011101100111	(union)

Boolean and Comparison Instructions

- 5. XOR Instruction

- The XOR instruction performs a boolean exclusive-OR operation between each pair of matching bits in two operands and stores the result in the destination operand:

XOR destination, source

- The XOR instruction uses the same operand combinations and sizes as the AND and OR instructions
- For each matching bit in the two operands, the following applies
 - If both bits are the same (both 0 or both 1), the result is 0; otherwise, the result is 1
- A bit exclusive-ORed with 0 retains its value, and a bit exclusive-ORed with 1 is toggled (complemented)
- XOR reverses itself when applied twice to the same operand
- The truth table shows that when bit x is exclusive-ORed with bit y twice, it reverts to its original value
 - This “reversible” property of XOR makes it an ideal tool for a simple form of symmetric encryption

x	y	$x \oplus y$	$(x \oplus y) \oplus y$
0	0	0	0
0	1	1	0
1	0	1	1
1	1	0	1

Boolean and Comparison Instructions

- 5. XOR Instruction

- Flags

- The XOR instruction always clears the Overflow and Carry flags
 - XOR modifies the Sign, Zero, and Parity flags according to the destination operand

- Checking the Parity Flag

- Parity checking is performed on a binary number that counts the number of 1 bits contained in the number
 - If the resulting count is even, we say that the data has even parity; if the count is odd, the data has odd parity
 - In x86 processors, the Parity flag is set when the lowest byte of the destination operand of a bitwise or arithmetic operation has even parity
 - Conversely, when the operand has odd parity, the flag is cleared
 - An effective way to check the parity of a number without changing its value is to exclusive-OR the number with zero

```
mov  al,10110101b    ; 5 bits = odd parity
xor  al,0             ; Parity flag clear (odd)
mov  al,11001100b    ; 4 bits = even parity
xor  al,0             ; Parity flag set (even)
```

- 16-Bit Parity

- You can check the parity of a 16-bit integer by performing an XOR between the upper and lower bytes
 - The parity of this union will be the same as the parity of the entire 16-bit integer:

```
mov  ax,64C1h        ; 0110 0100 1100 0001
xor  ah,al            ; Parity flag set (even)
```

Boolean and Comparison Instructions

- 6. NOT Instruction

- The NOT instruction toggles (inverts) all bits in an operand
- The result is called the *one's complement*

`NOT reg`
`NOT mem`

- For example, the one's complement of F0h is 0Fh

```
mov al,11110000b  
not al           ; AL = 00001111b
```

- Flags
 - No flags are affected by the NOT instruction

- 7. TEST Instruction

- The TEST instruction performs an AND operation, but does not modify the destination operand
- The TEST instruction permits the same operand combinations as the AND instruction
- TEST is particularly valuable for finding out whether individual bits in an operand are set

Boolean and Comparison Instructions

- 7. TEST Instruction

- Example: Testing Multiple Bits

- The TEST instruction can check several bits at once
 - Suppose we want to know whether bit 0 or bit 3 is set in the AL register
 - We can use the following instruction to find this out:

```
test al,00001001b;  test bits 0 and 3
(The value 00001001 in this example is called a bit mask.)
```

- From the following example data sets, we can infer that the Zero flag is set only when all tested bits are clear:

```
0 0 1 0 0 1 0 1  <- input value
0 0 0 0 1 0 0 1  <- test value
0 0 0 0 0 0 0 1  <- result: ZF = 0

0 0 1 0 0 1 0 0  <- input value
0 0 0 0 1 0 0 1  <- test value
0 0 0 0 0 0 0 0  <- result: ZF = 1
```

- Flags

- The TEST instruction always clears the Overflow and Carry flags
 - It modifies the Sign, Zero, and Parity flags in the same way as the AND instruction

Boolean and Comparison Instructions

- 8. CMP Instruction

- The most common boolean expressions involve some type of comparison

```
if A > B ...  
while X > 0 and X < 200 ...  
if check_for_error( N ) = true
```

- We use the CMP instruction to compare integers
 - Character codes are also integers, so they work with CMP as well
- CMP performs an implied subtraction of a source operand from a destination operand
 - Neither operand is modified
 - CMP uses the same operand combinations as the AND instruction

CMP destination, source

- Flags

- CMP changes the Overflow, Sign, Zero, Carry, Auxiliary Carry, and Parity flags according to the value the destination operand would have had if actual subtraction had taken place

unsigned operands

CMP Results	ZF	CF
Destination < source	0	1
Destination > source	0	0
Destination = source	1	0

signed operands

CMP Results	Flags
Destination < source	SF ≠ OF
Destination > source	SF = OF
Destination = source	ZF = 1

Boolean and Comparison Instructions

- 8. CMP Instruction

- Examples: how flags are affected by the CMP instruction

- When AX equals 5 and is compared to 10, the Carry flag is set because subtracting 10 from 5 requires a borrow:

```
mov ax,5
cmp ax,10                      ; ZF = 0 and CF = 1
```

- Comparing 1000 to 1000 sets the Zero flag because subtracting the source from the destination produces zero:

```
mov ax,1000
mov cx,1000
cmp cx,ax                      ; ZF = 1 and CF = 0
```

- Comparing 105 to 0 clears both the Zero and Carry flags because subtracting 0 from 105 generates a positive, nonzero value:

```
mov si,105
cmp si,0                       ; ZF = 0 and CF = 0
```

Boolean and Comparison Instructions

- 9. Setting and Clearing Individual CPU Flags

- How can you easily set or clear the Zero, Sign, Carry, and Overflow flags?
- There are several ways, some of which require modifying the destination operand
 - To set the Zero flag, TEST or AND an operand with Zero. To clear the Zero flag, OR an operand with 1:

```
test  al,0          ; set Zero flag
and   al,0          ; set Zero flag
or    al,1          ; clear Zero flag
```

- TEST does not modify the operand, whereas AND does
 - To set the Sign flag, OR the highest bit of an operand with 1. To clear the Sign flag, AND the highest bit with 0:

```
or    al,80h        ; set Sign flag
and   al,7Fh        ; clear Sign flag
```

- To set the Carry flag, use the STC instruction; to clear the Carry flag, use CLC:

```
stc                    ; set Carry flag
clc                    ; clear Carry flag
```

- To set the Overflow flag, add two positive values that produce a negative sum. To clear the Overflow flag, OR an operand with 0:

```
mov  al,7Fh          ; AL = +127
inc  al              ; AL = 80h (-128), OF=1
or   eax,0           ; clear Overflow flag
```

Conditional Jumps

- 1. Conditional Structures

- There are no explicit high-level logic structures in the x86 instruction set, but you can implement them using a combination of comparisons and jumps
- Two steps:
 1. An operation such as CMP, AND, or SUB modifies the CPU status flags
 2. A conditional jump instruction tests the flags and causes a branch to a new address

- Example 1

- The CMP instruction compares EAX to Zero
- The JZ (Jump if zero) instruction jumps to label L1 if the Zero flag was set by the CMP instruction

```
    cmp    eax, 0
    jz     L1                ; jump if ZF = 1
    .
    .
```

- Example 2

- The AND instruction performs a bitwise AND on the DL register, affecting the Zero flag
- The JNZ (jump if not Zero) instruction jumps if the Zero flag is clear

```
    and    dl, 10110000b
    jnz    L2                ; jump if ZF = 0
    .
    .
L2:
```

Conditional Jumps

- 2. Jcond Instruction

- A *conditional jump instruction* branches to a destination label when a status flag condition is true
 - Otherwise, if the flag condition is false, the instruction immediately following the conditional jump is executed
- The syntax:

Jcond destination

- *cond* refers to a flag condition identifying the state of one or more flags

JC	Jump if carry (Carry flag set)
JNC	Jump if not carry (Carry flag clear)
JZ	Jump if zero (Zero flag set)
JNZ	Jump if not zero (Zero flag clear)

- CPU status flags are most commonly set by arithmetic, comparison, and boolean instructions
- Conditional jump instructions evaluate the flag states, using them to determine whether or not jumps should be taken

Conditional Jumps

- 2. Jcond Instruction

- Using the CMP Instruction

- Example 1

- Suppose you want to jump to label L1 when EAX equals 5
 - If EAX equals 5, the CMP instruction sets the Zero flag; then, the JE instruction jumps to L1 because the Zero flag is set:

```
cmp    eax, 5
je     L1                ; jump if equal
```

- The JE instruction always jumps based on the value of the Zero flag
 - If EAX were not equal to 5, CMP would clear the Zero flag, and the JE instruction would not jump

- Example 2

- The JL instruction jumps to label L1 because AX is less than 6

```
mov    ax, 5
cmp    ax, 6
jl     L1                ; jump if less
```

- Example 3

- The jump is taken because AX is greater than 4

```
mov    ax, 5
cmp    ax, 4
jg     L1                ; jump if greater
```

Conditional Jumps

- 3. Types of Conditional Jump Instructions
 - Conditional jump instructions are able to compare signed and unsigned integers and perform actions based on the values of individual CPU flags
 - The conditional jump instructions can be divided into four groups:
 - Jumps based on specific flag values
 - Jumps based on equality between operands or the value of (E)CX
 - Jumps based on comparisons of unsigned operands
 - Jumps based on comparisons of signed operands
 - Jumps based on specific flag values

Mnemonic	Description	Flags / Registers
JZ	Jump if zero	ZF = 1
JNZ	Jump if not zero	ZF = 0
JC	Jump if carry	CF = 1
JNC	Jump if not carry	CF = 0
JO	Jump if overflow	OF = 1
JNO	Jump if not overflow	OF = 0
JS	Jump if signed	SF = 1
JNS	Jump if not signed	SF = 0
JP	Jump if parity (even)	PF = 1
JNP	Jump if not parity (odd)	PF = 0

Conditional Jumps

- 3. Types of Conditional Jump Instructions
 - Equality Comparisons

Mnemonic	Description
JE	Jump if equal (<i>leftOp</i> = <i>rightOp</i>)
JNE	Jump if not equal (<i>leftOp</i> $\neq$ <i>rightOp</i>)
JCXZ	Jump if CX = 0
JECXZ	Jump if ECX = 0
JRCXZ	Jump if RCX = 0 (64-bit mode)

- In some cases, two operands are compared; in other cases, a jump is taken based on the value of (E)CX
- The notations *leftOp* and *rightOp* refer to the left (destination) and right (source) operands in a CMP instruction:

CMP *leftOp, rightOp*

- The operand names reflect the ordering of operands for relational operators in algebra
 - For example, in the expression $X < Y$, X is called *leftOp* and Y is called *rightOp*
- The JE instruction is equivalent to JZ and JNE is equivalent to JNZ
 - It's best to select the mnemonic (JE or JZ) that best indicates your intention to either compare two operands or examine a specific status flag

Conditional Jumps

- 3. Types of Conditional Jump Instructions
 - Equality Comparisons

- Example 1

```
mov  edx, 0A523h
cmp  edx, 0A523h
jne  L5           ; jump not taken
je   L1           ; jump is taken
```

- Example 2

```
mov  bx, 1234h
sub  bx, 1234h
jne  L5           ; jump not taken
je   L1           ; jump is taken
```

- Example 3

```
mov  cx, 0FFFFh
inc  cx
jcxz L2           ; jump is taken
```

- Example 4

```
xor  ecx, ecx
jecxz L2          ; jump is taken
```

Conditional Jumps

- 3. Types of Conditional Jump Instructions
 - Unsigned Comparisons
 - These jumps are only meaningful when comparing unsigned values

Mnemonic	Description
JA	Jump if above (if $leftOp > rightOp$)
JNBE	Jump if not below or equal (same as JA)
JAE	Jump if above or equal (if $leftOp \geq rightOp$)
JNB	Jump if not below (same as JAE)
JB	Jump if below (if $leftOp < rightOp$)
JNAE	Jump if not above or equal (same as JB)
JBE	Jump if below or equal (if $leftOp \leq rightOp$)
JNA	Jump if not above (same as JBE)

Conditional Jumps

- 3. Types of Conditional Jump Instructions
 - Signed Comparisons

Mnemonic	Description
JG	Jump if greater (if $leftOp > rightOp$)
JNLE	Jump if not less than or equal (same as JG)
JGE	Jump if greater than or equal (if $leftOp \geq rightOp$)
JNL	Jump if not less (same as JGE)
JL	Jump if less (if $leftOp < rightOp$)
JNGE	Jump if not greater than or equal (same as JL)
JLE	Jump if less than or equal (if $leftOp \leq rightOp$)
JNG	Jump if not greater (same as JLE)

```
mov  al,+127           ; hexadecimal value is 7Fh
cmp   al,-128          ; hexadecimal value is 80h
ja    IsAbove          ; jump not taken, because 7Fh < 80h
jg    IsGreater        ; jump taken, because +127 > -128
```

Conditional Jumps

- 3. Types of Conditional Jump Instructions

- Example 1

```
mov    edx, -1
cmp    edx, 0
jnl    L5           ; jump not taken (-1 >= 0 is false)
jnle   L5           ; jump not taken (-1 > 0 is false)
jl     L1           ; jump is taken (-1 < 0 is true)
```

- Example 2

```
mov    bx, +32
cmp    bx, -35
jng    L5           ; jump not taken (+32 <= -35 is false)
jnge   L5           ; jump not taken (+32 < -35 is false)
jge    L1           ; jump is taken (+32 >= -35 is true)
```

- Example 3

```
mov    ecx, 0
cmp    ecx, 0
jg     L5           ; jump not taken (0 > 0 is false)
jnl    L1           ; jump is taken (0 >= 0 is true)
```

- Example 4

```
mov    ecx, 0
cmp    ecx, 0
jl     L5           ; jump not taken (0 < 0 is false)
jng    L1           ; jump is taken (0 <= 0 is true)
```

Conditional Jumps

- 4. Conditional Jump Applications

- Testing Status Bits

- Often, we do not want to change the values of the bits we're testing, but we do want to modify the values of CPU status flags
 - Conditional jump instructions often use these status flags to determine whether or not to transfer control to code labels
 - Suppose that an 8-bit memory operand named *status* contains status information about an external device attached to the computer
 - The following instructions jump to a label if bit 5 is set, indicating that the device is offline:

```
mov    al,status
test   al,00100000b           ; test bit 5
jnz    DeviceOffline
```

- The following statements jump to a label if any of the bits 0, 1, or 4 are set:

```
mov    al,status
test   al,00010011b           ; test bits 0,1,4
jnz    InputDataByte
```

- Jumping to a label if bits 2, 3, and 7 are all set requires both the AND and CMP instructions:

```
mov    al,status
and     al,10001100b           ; mask bits 2,3,7
cmp     al,10001100b           ; all bits set?
je      ResetMachine           ; yes: jump to label
```

Conditional Jumps

- 4. Conditional Jump Applications

- Larger of Two Integers

- The following code compares the unsigned integers in EAX and EBX and moves the larger of the two to EDX:

```
        mov    edx,eax                ; assume EAX is larger
        cmp    eax,ebx                ; if EAX is >= EBX
        jae    L1                     ;   jump to L1
        mov    edx,ebx                ; else move EBX to EDX
L1:                                     ; EDX contains the larger integer
```

- Smallest of Three Integers

- The following instructions compare the unsigned 16-bit values in the variables V1, V2, and V3 and move the smallest of the three to AX:

```
.data
V1 WORD ?
V2 WORD ?
V3 WORD ?
.code
        mov    ax,V1                 ; assume V1 is smallest
        cmp    ax,V2                 ; if AX <= V2
        jbe    L1                     ;   jump to L1
        mov    ax,V2                 ; else move V2 to AX
L1:      cmp    ax,V3                 ; if AX <= V3
        jbe    L2                     ;   jump to L2
        mov    ax,V3                 ; else move V3 to AX
L2:
```

Conditional Jumps

- 4. Conditional Jump Applications

- Loop until Key Pressed

- In the following 32-bit code, a loop runs continuously until the user presses a standard alphanumeric key
 - The ReadKey method from the Irvine32 library sets the Zero flag if no key is present in the input buffer

```
.data
char BYTE ?
.code
L1: mov  eax,10           ; create 10 ms delay
    call Delay
    call ReadKey          ; check for key
    jz   L1               ; repeat if no key
    mov  char,AL          ; save the character
```

- The code inserts a 10-millisecond delay in the loop to give MS-Windows time to process event messages
 - If you omit the delay, keystrokes may be ignored

Conditional Jumps

- 4. Conditional Jump Applications
 - Application: Sequential Search of an Array
 - The following program looks for the first nonzero value in an array of 16-bit integers
 - If it finds one, it displays the value
 - Otherwise, it displays a message stating that a nonzero value was not found

```
; Scanning an Array                                (ArrayScan.asm)
; Scan an array for the first nonzero value.

INCLUDE Irvine32.inc

.data
intArray  SWORD  0,0,0,0,1,20,35,-12,66,4,0
;intArray SWORD  1,0,0,0                        ; alternate test data
;intArray SWORD  0,0,0,0                        ; alternate test data
;intArray SWORD  0,0,0,1                        ; alternate test data
noneMsg   BYTE  "A non-zero value was not found",0

.code
main PROC
    mov  ebx,OFFSET intArray      ; point to the array
    mov  ecx,LENGTHOF intArray    ; loop counter
```

```
L1:  cmp    WORD PTR [ebx],0      ; compare value to zero
     jnz    found                ; found a value
     add    ebx,2                ; point to next
     loop   L1                   ; continue the loop
     jmp    notFound            ; none found

found:                                ; display the value
     movsx  eax,WORD PTR [ebx]    ; sign-extend into EAX
     call   WriteInt
     jmp    quit

notFound:                                ; display "not found" message
     mov    edx,OFFSET noneMsg
     call   WriteString

quit:
     call   Crlf
     exit
main ENDP
END main
```

Conditional Jumps

- 4. Conditional Jump Applications

- Application: Simple String Encryption

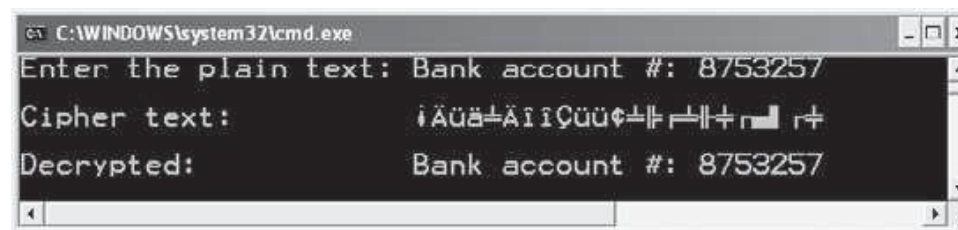
- If an integer X is XORed with Y and the resulting value is XORed with Y again, the value produced is X:

$$((X \otimes Y) \otimes Y) = X$$

- This reversible property of XOR provides an easy way to perform a simple form of data encryption:
 - » A *plain text* message is transformed into an encrypted string called *cipher text* by XORing each of its characters with a character from a third string called a *key*
 - » The intended viewer can use the key to decrypt the cipher text and produce the original plain text

- Example Program

- We will demonstrate a simple program that uses symmetric encryption, a process by which the same key is used for both encryption and decryption
 - The following steps occur in order at runtime:
 1. The user enters the plain text
 2. The program uses a single-character key to encrypt the plain text, producing the cipher text, which is displayed on the screen
 3. The program decrypts the cipher text, producing and displaying the original plain text
 - Here is sample output from the program



```
C:\WINDOWS\system32\cmd.exe
Enter the plain text: Bank account #: 8753257
Cipher text:      iÄüä±Äi1Qüü±±±±±±±±±±
Decrypted:      Bank account #: 8753257
```

```

INCLUDE Irvine32.inc
KEY = 239                ; any value between 1-255
BUFMAX = 128             ; maximum buffer size

.data
sPrompt BYTE "Enter the plain text:",0
sEncrypt BYTE "Cipher text: ",0
sDecrypt BYTE "Decrypted: ",0
buffer BYTE BUFMAX+1 DUP(0)
bufSize DWORD ?

.code
main PROC
    call InputTheString    ; input the plain text
    call TranslateBuffer   ; encrypt the buffer
    mov  edx,OFFSET sEncrypt ; display encrypted message
    call DisplayMessage
    call TranslateBuffer   ; decrypt the buffer
    mov  edx,OFFSET sDecrypt ; display decrypted message
    call DisplayMessage
    exit
main ENDP

;-----
InputTheString PROC
;
; Prompts user for a plaintext string. Saves the string
; and its length.
; Receives: nothing
; Returns: nothing
;-----
    pushad                ; save 32-bit registers
    mov  edx,OFFSET sPrompt ; display a prompt
    call WriteString
    mov  ecx,BUFMAX        ; maximum character count
    mov  edx,OFFSET buffer ; point to the buffer
    call ReadString        ; input the string
    mov  bufSize,eax       ; save the length
    call Crlf

```

```

    popad
    ret
InputTheString ENDP

;-----
DisplayMessage PROC
;
; Displays the encrypted or decrypted message.
; Receives: EDX points to the message
; Returns: nothing
;-----
    pushad
    call WriteString
    mov  edx,OFFSET buffer ; display the buffer
    call WriteString
    call Crlf
    call Crlf
    popad
    ret
DisplayMessage ENDP

;-----
TranslateBuffer PROC
;
; Translates the string by exclusive-ORing each
; byte with the encryption key byte.
; Receives: nothing
; Returns: nothing
;-----
    pushad
    mov  ecx,bufSize      ; loop counter
    mov  esi,0            ; index 0 in buffer
L1:
    xor  buffer[esi],KEY   ; translate a byte
    inc  esi              ; point to next byte
    loop L1
    popad
    ret
TranslateBuffer ENDP
END main

```

Conditional Loop Instructions

- 1. LOOPZ and LOOPE Instructions
 - The LOOPZ (loop if zero) instruction
 - This instruction works like the LOOP instruction, but the Zero flag must be set in order for control to transfer to the destination label

`LOOPZ destination`

- The LOOPE (loop if equal) instruction
 - Equivalent to LOOPZ, and they share the same opcode

- They perform the following tasks:

```
ECX = ECX - 1
if ECX > 0 and ZF = 1, jump to destination
```

- Otherwise, no jump occurs, and control passes to the next instruction
- LOOPZ and LOOPE do not affect any of the status flags

Conditional Loop Instructions

- 2. LOOPNZ and LOOPNE Instructions

- The LOOPNZ (loop if not zero) instruction

- The counterpart of LOOPZ
 - The loop continues while the unsigned value of ECX is greater than zero (after being decremented) and the Zero flag is clear

`LOOPNZ destination`

- The LOOPNE (loop if not equal) instruction

- Equivalent to LOOPNZ, and they share the same opcode

- They perform the following tasks:

```
ECX = ECX - 1
if ECX > 0 and ZF = 0, jump to destination
```

- Otherwise, nothing happens, and control passes to the next instruction

Conditional Loop Instructions

- 2. LOOPNZ and LOOPNE Instructions

- Example

- The following code scans each number in an array until a nonnegative number is found
 - Notice that we push the flags on the stack before the ADD instruction because ADD will modify the flags
 - Then the flags are restored by POPFD just before the LOOPNZ instruction executes

```
.data
array  SWORD  -3,-6,-1,-10,10,30,40,4
sentinel SWORD  0
.code
    mov     esi,OFFSET array
    mov     ecx,LENGTHOF array

L1:  test    WORD PTR [esi],8000h          ; test sign bit
     pushfd                                ; push flags on stack
     add     esi,TYPE array                ; move to next position
     popfd                                 ; pop flags from stack
     loopnz  L1                           ; continue loop
     jnz     quit                          ; none found
     sub     esi,TYPE array                ; ESI points to value
quit:
```

Conditional Structures

- Conditional structure
 - One or more conditional expressions that trigger a choice between different logical branches
 - Each branch causes a different sequence of instructions to execute

- 1. Block-Structured IF Statements

- If structure in HLL

```
if( boolean-expression )  
    statement-list-1  
else  
    statement-list-2
```

- In assembly language
 - First, we evaluate the boolean expression in such a way that one of the CPU status flags is affected
 - Second, we construct a series of jumps that transfer control to the two lists of statements, based on the value of the relevant CPU status flag

Conditional Structures

- 1. Block-Structured IF Statements
 - Example 1

```
if( op1 == op2 )
{
    X = 1;
    Y = 2;
}
```

- We can translate this IF statement into assembly language with a CMP instruction followed by conditional jumps

```
mov    eax,op1
cmp    eax,op2                ; op1 == op2?
jne    L1                    ; no: skip next
mov    X,1                    ; yes: assign X and Y
mov    Y,2
L1:
```

- If we implemented the == operator using JE, the resulting code would be slightly less compact (six instructions rather than five)

```
mov    eax,op1
cmp    eax,op2                ; op1 == op2?
je     L1                    ; yes: jump to L1
jmp    L2                    ; no: skip assignments
L1:    mov    X,1                ; assign X and Y
      mov    Y,2
L2:
```

Conditional Structures

- 1. Block-Structured IF Statements

- Example 2

- In the NTFS file storage system, the size of a disk cluster depends on the disk volume's overall capacity
 - In the following pseudocode, we set the cluster size to 4,096 if the volume size is less than 16 Tbytes
 - Otherwise, we set the cluster size to 8,192

```
clusterSize = 8192;  
if terrabytes < 16  
    clusterSize = 4096;
```

- We can implement the pseudocode in assembly language

```
mov    clusterSize,8192    ; assume larger cluster  
cmp    terrabytes, 16      ; smaller than 16 TB?  
jae    next  
mov    clusterSize,4096    ; switch to smaller cluster  
next:
```

Conditional Structures

- 1. Block-Structured IF Statements
 - Example 3

```
if op1 > op2
    call Routine1
else
    call Routine2
end if
```

- In the following assembly language translation of the pseudocode, we assume that op1 and op2 are signed doubleword variables
- When comparing variables, one must be moved to a register

```
mov    eax,op1                ; move op1 to a register
cmp    eax,op2                ; op1 > op2?
jg     A1                     ; yes: call Routine1
call   Routine2               ; no: call Routine2
jmp    A2                     ; exit the IF statement
A1:    call Routine1
A2:
```

Conditional Structures

- 1. Block-Structured IF Statements
 - White Box Testing
 - Complex conditional statements may have multiple execution paths, making them hard to debug
 - Programmers often implement a technique known as white box testing, which verifies a subroutine's inputs and corresponding outputs
 - White box testing requires you to have a copy of the source code
 - You assign a variety of values to the input variables
 - For each combination of inputs, you manually trace through the source code and verify the execution path and outputs produced by the subroutine

```
if op1 == op2
  if X > Y
    call Routine1
  else
    call Routine2
  end if
else
  call Routine3
end if
```

Conditional Structures

- 1. Block-Structured IF Statements

- White Box Testing

- The following code reverses the initial condition ($op1 == op2$) and immediately jumps to the ELSE portion
 - All that is left to translate is the inner IF-ELSE statement

```
1:      mov    eax,op1
2:      cmp    eax,op2                ; op1 == op2?
3:      jne    L2                    ; no: call Routine3

; process the inner IF-ELSE statement.
4:      mov    eax,X
5:      cmp    eax,Y                ; X > Y?
6:      jg     L1                    ; yes: call Routine1
7:      call   Routine2              ; no: call Routine2
8:      jmp    L3                    ; and exit
9:  L1: call   Routine1              ; call Routine1
10:     jmp    L3                    ; and exit
11:  L2: call   Routine3
12:  L3:
```

op1	op2	X	Y	Line Execution Sequence	Calls
10	20	30	40	1, 2, 3, 11, 12	Routine3
10	20	40	30	1, 2, 3, 11, 12	Routine3
10	10	30	40	1, 2, 3, 4, 5, 6, 7, 8, 12	Routine2
10	10	40	30	1, 2, 3, 4, 5, 6, 9, 10, 12	Routine1

Conditional Structures

- 2. Compound Expressions
 - Logical AND Operator

```
if (a1 > b1) AND (b1 > c1)
    X = 1
end if
```

- Short-Circuit Evaluation
 - The following is a straightforward implementation using short-circuit evaluation
 - » The second expression is not evaluated if the first expression is false

```
        cmp    a1,b1                ; first expression...
        ja     L1
        jmp    next
L1:      cmp    b1,c1                ; second expression...
        ja     L2
        jmp    next
L2:      mov    X,1                  ; both true: set X to 1
next:
```

- We can reduce the code to five instructions by changing the initial JA instruction to JBE

```
        cmp    a1,b1                ; first expression...
        jbe    next                 ; quit if false
        cmp    b1,c1                ; second expression
        jbe    next                 ; quit if false
        mov    X,1                  ; both are true
next:
```

Conditional Structures

- 2. Compound Expressions
 - Logical OR Operator

```
if (a1 > b1) OR (b1 > c1)  
    X = 1
```

- In the following implementation, the code branches to L1 if the first expression is true
 - otherwise, it falls through to the second CMP instruction
- The second expression reverses the > operator and uses JBE instead

```
        cmp    al,b1                ; 1: compare AL to BL  
        ja     L1                   ; if true, skip second expression  
        cmp    bl,cl                ; 2: compare BL to CL  
        jbe     next                ; false: skip next statement  
L1:      mov    X,1                  ; true: set X = 1  
next:
```

Conditional Structures

- 3. WHILE Loops

```
while( val1 < val2 )
{
    val1++;
    val2--;
}
```

- It is convenient to reverse the loop condition and jump to `endwhile` if a condition becomes true
- Assuming that `val1` and `val2` are variables, we must copy one of them to a register at the beginning and restore the variable's value at the end

```
        mov    eax, val1                ; copy variable to EAX
beginwhile:
        cmp    eax, val2                ; if not (val1 < val2)
        jnl    endwhile                ; exit the loop
        inc    eax                      ; val1++;
        dec    val2                     ; val2--;
        jmp    beginwhile               ; repeat the loop
endwhile:
        mov    val1, eax                ; save new value for val1
```

- EAX is a substitute for `val1` inside the loop
- References to `val1` must be through EAX
- JNL is used, implying that `val1` and `val2` are signed integers

Conditional Structures

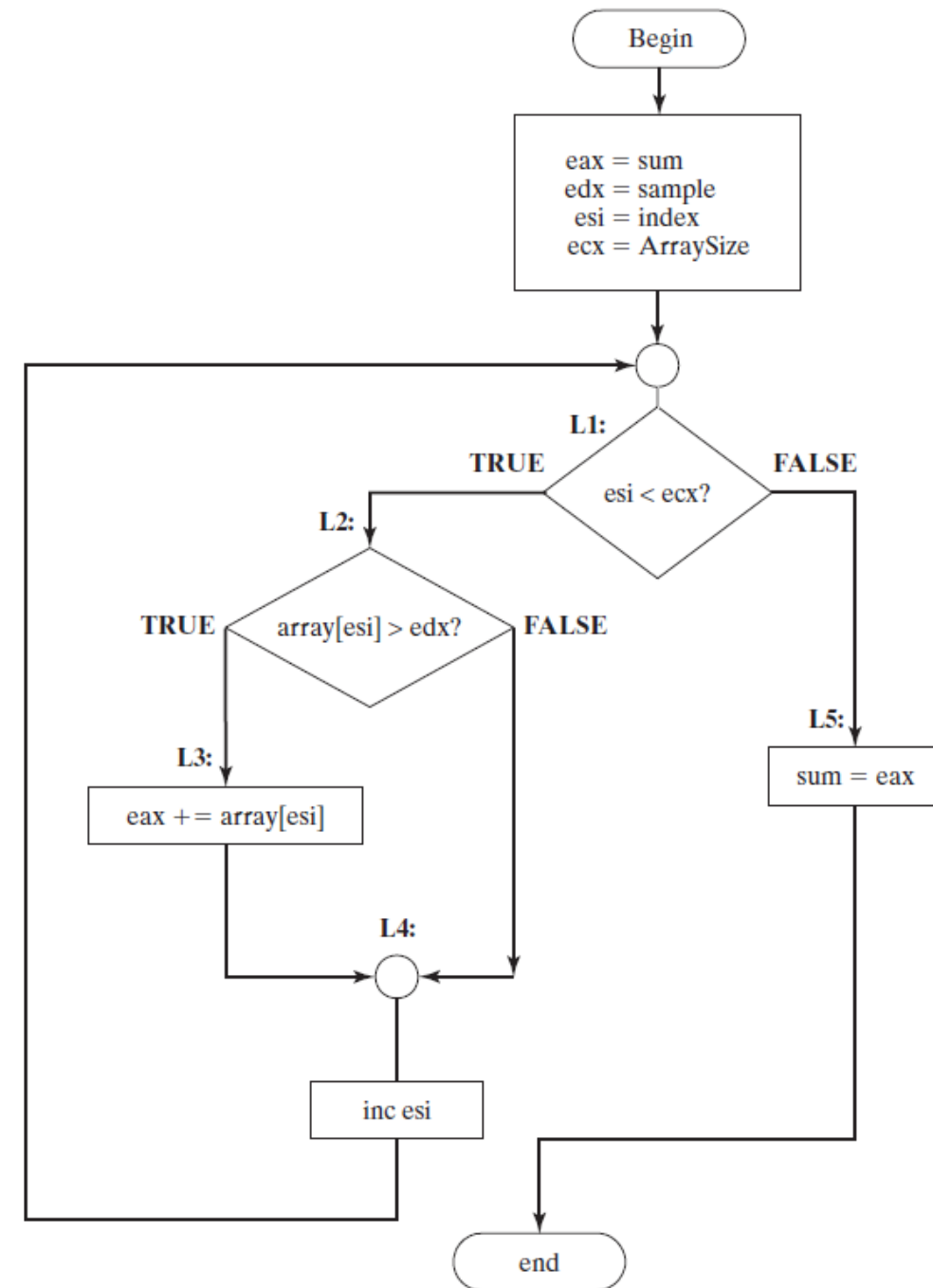
• 3. WHILE Loops

– Example: IF statement Nested in a Loop

- The following code calculates the sum of all array elements greater than the value in sample

```
int array[] = {10,60,20,33,72,89,45,65,72,18};  
int sample = 50;  
int ArraySize = sizeof array / sizeof sample;  
int index = 0;  
int sum = 0;  
while( index < ArraySize )  
{  
    if( array[index] > sample )  
    {  
        sum += array[index];  
    }  
    index++;  
}
```

- To simplify the translation and speed up execution by reducing the number of memory accesses, registers have been substituted for variables
 - EDX = sample, EAX = sum, ESI = index, and ECX = ArraySize



Conditional Structures

- 3. WHILE Loops
 - Example: IF statement Nested in a Loop
 - Assembly Code

```
.data
sum DWORD 0
sample DWORD 50
array DWORD 10,60,20,33,72,89,45,65,72,18
ArraySize = ($ - Array) / TYPE array

.code
main PROC
    mov     eax,0                ; sum
    mov     edx,sample
    mov     esi,0                ; index
    mov     ecx,ArraySize
```

```
L1:  cmp     esi,ecx              ; if esi < ecx
      jl     L2
      jmp     L5

L2:  cmp     array[esi*4], edx    ; if array[esi] > edx
      jg     L3
      jmp     L4

L3:  add     eax,array[esi*4]

L4:  inc     esi
      jmp     L1

L5:  mov     sum,eax
```

Conditional Structures

- 4. Table-Driven Selection

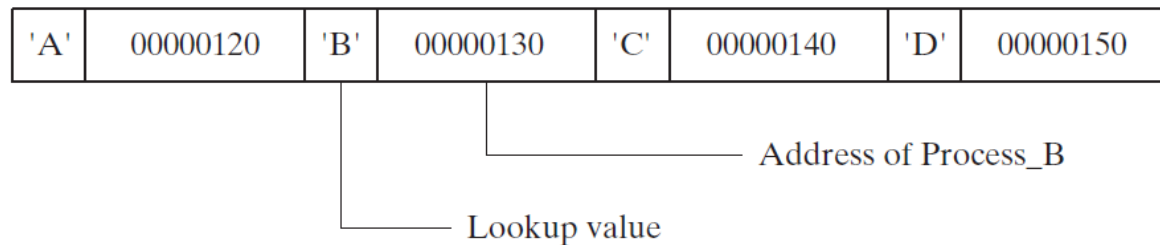
- A way of using a table lookup to replace a multiway selection structure

- You must create a lookup table, and then you must use a loop to search the table
 - The table must contain lookup values and the offsets of labels or procedures

- E.g.) The following is part of a table containing single-character lookup values and addresses of procedures

```
.data
CaseTable BYTE    'A'                ; lookup value
          DWORD Process_A            ; address of procedure
          BYTE    'B'
          DWORD Process_B
          (etc.)
```

- Assume that Process_A, Process_B, Process_C, and Process_D are located at addresses 120h, 130h, 140h, and 150h, respectively



Conditional Structures

- 4. Table-Driven Selection

- Example Program

- The user inputs a character from the keyboard
 - Using a loop, the character is compared to each entry in a lookup table
 - The 1st match found causes a call to the procedure offset stored immediately after the lookup value
 - Each procedure loads EDX with the offset of a different string, which is displayed during the loop

```
INCLUDE Irvine32.inc
.data
CaseTable BYTE 'A'           ; lookup value
          DWORD  Process_A   ; address of procedure
EntrySize = ($ - CaseTable)
          BYTE 'B'
          DWORD  Process_B
          BYTE 'C'
          DWORD  Process_C
          BYTE 'D'
          DWORD  Process_D
NumberOfEntries = ($ - CaseTable) / EntrySize
prompt BYTE "Press capital A,B,C,or D: ",0
msgA BYTE "Process_A",0
msgB BYTE "Process_B",0
msgC BYTE "Process_C",0
msgD BYTE "Process_D",0
```

```
.code
main PROC
    mov     edx,OFFSET prompt    ; ask user for input
    call    WriteString
    call    ReadChar             ; read character into AL
    mov     ebx,OFFSET CaseTable ; point EBX to the table
    mov     ecx,NumberOfEntries ; loop counter

L1:
    cmp     al,[ebx]              ; match found?
    jne     L2                   ; no: continue
    call    NEAR PTR [ebx + 1]    ; yes: call the procedure
    call    WriteString           ; display message
    call    Crlf
    jmp     L3                   ; exit the search

L2:
    add     ebx,EntrySize         ; point to the next entry
    loop    L1                   ; repeat until ECX = 0

L3:
    exit
main ENDP
```

```
Process_A PROC
    mov     edx,OFFSET msgA
    ret
Process_A ENDP

Process_B PROC
    mov     edx,OFFSET msgB
    ret
Process_B ENDP

Process_C PROC
    mov     edx,OFFSET msgC
    ret
Process_C ENDP

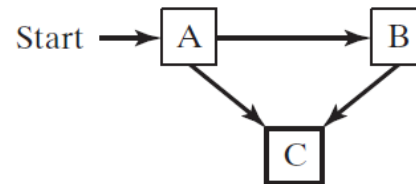
Process_D PROC
    mov     edx,OFFSET msgD
    ret
Process_D ENDP
END main
```

Conditional Structures

- 4. Table-Driven Selection
 - Cons
 - The table-driven selection method involves some initial overhead
 - Pros
 - The table-driven selection method can reduce the amount of code you write
 - A table can handle a large number of comparisons, and it can be more easily modified than a long series of compare, jump, and CALL instructions
 - A table can even be reconfigured at runtime

Application: Finite-State Machines

- A finite-state machine (FSM)
 - A machine or program that changes state based on some input
 - A graph which represents an FSM
 - A *node*: a program state
 - An *edge*: a transition from one state to another
 - One node is designated as the *initial state*
 - One or more states are designated as *terminal states*
 - A terminal state represents a state in which the program might stop without producing an error
 - A FSM is a specific instance of a more general type of structure called a *directed graph*



Application: Finite-State Machines

- 1. Validating an Input String
 - Programs that read input streams often must validate their input
 - E.g.) A programming language compiler can use a FSM to scan source programs and convert words and symbols into tokens, which are usually keywords, arithmetic operators, and identifiers
 - A FSM for checking the validity of an input string
 - Read the input character by character
 - Each character is represented by an edge in the diagram
 - A FSM detects illegal input sequences in one of two ways:
 - The next input character does not correspond to any transitions from the current state
 - The end of input is reached and the current state is a nonterminal state

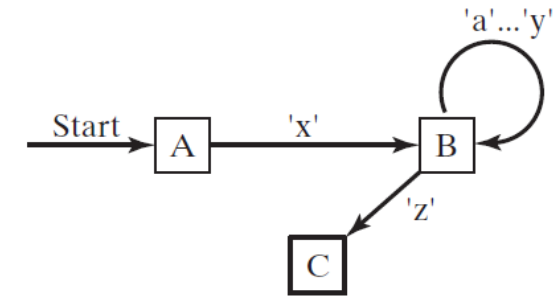
Application: Finite-State Machines

- 1. Validating an Input String

- Character String Example

- Let's check the validity of an input string according to the following two rules:
 - The string must begin with the letter "x" and end with the letter "z"
 - Between the first and last characters, there can be zero or more letters within the range {'a'...'y'}
- Each transition is identified with a particular type of input
 - The transition from state A to state B can only be accomplished if the letter x is read from the input stream
 - A transition from state B to itself is accomplished by the input of any letter of the alphabet except z
 - A transition from state B to state C occurs only when the letter z is read from the input stream
- If the end of the input stream is reached while the program is in state A or B, an error condition results because only state C is marked as a terminal state
- The following input strings would be recognized by this FSM:

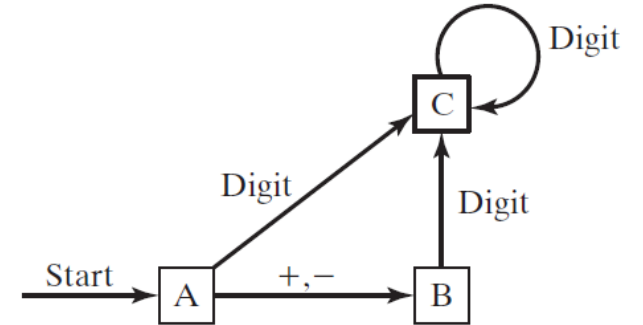
`xaabcdefgz`
`xz`
`xyyqqrstuvz`



Application: Finite-State Machines

- 2. Validating a Signed Integer

- Input consists of an optional leading sign followed by a sequence of digits
- There is no maximum number of digits implied by the diagram



- Translating finite-state machines into assembly language code

- Each state in the diagram (A, B, C, ...) is represented in the program by a label
- The following actions are performed at each label
 1. A call to an input procedure reads the next character from input
 2. If the state is a terminal state, check to see whether the user has pressed the Enter key to end the input
 3. One or more compare instructions check for each possible transition leading away from the state. Each comparison is followed by a conditional jump instruction
- E.g.) At state A, the following code reads the next input character and checks for a possible transition to state B

StateA:

```
call  Getnext           ; read next char into AL
cmp   al, '+'           ; leading + sign?
je    StateB            ; go to State B
cmp   al, '-'           ; leading - sign?
je    StateB            ; go to State B
call  IsDigit           ; ZF = 1 if AL contains a digit
jz    StateC            ; go to State C
call  DisplayErrorMsg   ; invalid input found
jmp   Quit
```

Application: Finite-State Machines

- 2. Validating a Signed Integer
 - Complete Finite-State Machine Program

```
INCLUDE Irvine32.inc

ENTER_KEY = 13

.data
InvalidInputMsg BYTE "Invalid input",13,10,0

.code
main PROC
    call Clrscr

StateA:
    call Getnext                ; read next char into AL
    cmp al,'+'                  ; leading + sign?
    je StateB                   ; go to State B
    cmp al,'-'                  ; leading - sign?
    je StateB                   ; go to State B
    call IsDigit                ; ZF = 1 if AL contains a digit
    jz StateC                   ; go to State C
    call DisplayErrorMsg        ; invalid input found
    jmp Quit

StateB:
    call Getnext                ; read next char into AL
    call IsDigit                ; ZF = 1 if AL contains a digit
    jz StateC                   ; go to State C
    call DisplayErrorMsg        ; invalid input found
    jmp Quit
```

```
StateC:
    call Getnext                ; read next char into AL
    call IsDigit                ; ZF = 1 if AL contains a digit
    jz StateC
    cmp al,ENTER_KEY           ; Enter key pressed?
    je Quit                    ; yes: quit
    call DisplayErrorMsg        ; no: invalid input found
    jmp Quit
```

```
Quit:
    call Crlf
    exit

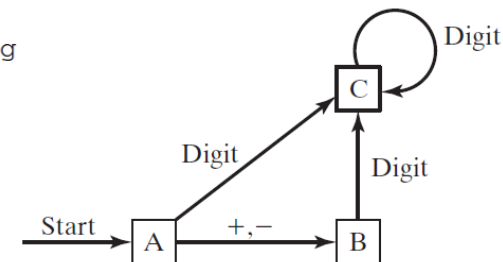
main ENDP

Getnext PROC
;
; Reads a character from standard input.
; Receives: nothing
; Returns: AL contains the character
;-----
    call ReadChar              ; input from keyboard
    call WriteChar              ; echo on screen
    ret
```

```
Getnext ENDP
```

```
DisplayErrorMsg PROC
;
; Displays an error message indicating that
; the input stream contains illegal input.
; Receives: nothing.
; Returns: nothing
;-----
```

```
    push edx
    mov edx,OFFSET InvalidInputMsg
    call WriteString
    pop edx
    ret
DisplayErrorMsg ENDP
END main
```



Application: Finite-State Machines

- 2. Validating a Signed Integer

- IsDigit Procedure

- The Finite-State Machine sample program calls the IsDigit procedure, which belongs to the Irvine32 library
 - IsDigit receives the AL register as input, and the value it returns is the setting of the Zero flag

```
-----  
IsDigit PROC  
;  
; Determines whether the character in AL is a valid decimal digit.  
; Receives: AL = character  
; Returns: ZF = 1 if AL contains a valid decimal digit; otherwise, ZF = 0.  
-----  
        cmp     al, '0'  
        jnb     ID1             ; ZF = 0 when jump taken  
        cmp     al, '9'  
        ja      ID1             ; ZF = 0 when jump taken  
        test    ax, 0           ; set ZF = 1  
ID1: ret  
IsDigit ENDP
```

Conditional Control Flow Directives

- MASM includes a number of high-level conditional control flow directives that help to simplify the coding of conditional statements
- In the preprocessing step, the assembler recognizes directives such as .CODE, .DATA, as well as *conditional control flow directives*

Directive	Description
.BREAK	Generates code to terminate a .WHILE or .REPEAT block
.CONTINUE	Generates code to jump to the top of a .WHILE or .REPEAT block
.ELSE	Begins block of statements to execute when the .IF condition is false
.ELSEIF <i>condition</i>	Generates code that tests <i>condition</i> and executes statements that follow, until an .ENDIF directive or another .ELSEIF directive is found
.ENDIF	Terminates a block of statements following an .IF, .ELSE, or .ELSEIF directive
.ENDW	Terminates a block of statements following a .WHILE directive
.IF <i>condition</i>	Generates code that executes the block of statements if <i>condition</i> is true.
.REPEAT	Generates code that repeats execution of the block of statements until <i>condition</i> becomes true
.UNTIL <i>condition</i>	Generates code that repeats the block of statements between .REPEAT and .UNTIL until <i>condition</i> becomes true
.UNTILCXZ	Generates code that repeats the block of statements between .REPEAT and .UNTILCXZ until CX equals zero
.WHILE <i>condition</i>	Generates code that executes the block of statements between .WHILE and .ENDW as long as <i>condition</i> is true

Conditional Control Flow Directives

- 1. Creating IF Statements

- .IF, .ELSE, .ELSEIF, and .ENDIF directives

- They make it easy for you to code multiway branching logic
 - They cause the assembler to generate CMP and conditional jump instructions in the background, which appear in the output listing file

```
.IF condition1
    statements
[.ELSEIF condition2
    statements ]
[.ELSE
    statements ]
.ENDIF
```

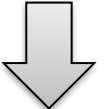
- The square brackets show that .ELSEIF and .ELSE are optional, whereas .IF and .ENDIF are required
 - A condition is a boolean expression involving the same operators used in C (such as <, >, ==, and !=)
 - The expression is evaluated at runtime

```
(eax > 0) && (eax > 10000h)
(val11 <= 100) || (val12 <= 100)
(val12 != ebx) && !CARRY?
```

Conditional Control Flow Directives

- 1. Creating IF Statements
 - Generating ASM Code

```
mov eax,6
.IF eax > val1
    mov result,1
.ENDIF
```



```
mov    eax,6
cmp    eax,val1
jbe    @C0001
mov    result,1
@C0001:
```

Operator	Description
<i>expr1</i> == <i>expr2</i>	Returns true when <i>expr1</i> is equal to <i>expr2</i> .
<i>expr1</i> != <i>expr2</i>	Returns true when <i>expr1</i> is not equal to <i>expr2</i> .
<i>expr1</i> > <i>expr2</i>	Returns true when <i>expr1</i> is greater than <i>expr2</i> .
<i>expr1</i> ≥ <i>expr2</i>	Returns true when <i>expr1</i> is greater than or equal to <i>expr2</i> .
<i>expr1</i> < <i>expr2</i>	Returns true when <i>expr1</i> is less than <i>expr2</i> .
<i>expr1</i> ≤ <i>expr2</i>	Returns true when <i>expr1</i> is less than or equal to <i>expr2</i> .
! <i>expr</i>	Returns true when <i>expr</i> is false.
<i>expr1</i> && <i>expr2</i>	Performs logical AND between <i>expr1</i> and <i>expr2</i> .
<i>expr1</i> <i>expr2</i>	Performs logical OR between <i>expr1</i> and <i>expr2</i> .
<i>expr1</i> & <i>expr2</i>	Performs bitwise AND between <i>expr1</i> and <i>expr2</i> .
CARRY?	Returns true if the Carry flag is set.
OVERFLOW?	Returns true if the Overflow flag is set.
PARITY?	Returns true if the Parity flag is set.
SIGN?	Returns true if the Sign flag is set.
ZERO?	Returns true if the Zero flag is set.

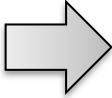
- You can view expanded assembly language instructions if you run the program in the Visual Studio debugger, right-click, and select Go to Disassembly

Conditional Control Flow Directives

- 2. Signed and Unsigned Comparisons

- When you use the .IF directive to compare values, you must be aware of how MASM generates conditional jumps
 - If the comparison involves an unsigned variable, an unsigned conditional jump instruction is inserted in the generated code

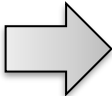
```
.data
val1 DWORD 5
result DWORD ?
.code
    mov eax,6
    .IF eax > val1
        mov result,1
    .ENDIF
```



```
    mov eax,6
    cmp eax,val1
    jbe @C0001
    mov result,1
@C0001:
```

- Comparing a Signed Integer

```
.data
val2 SDWORD -1
result DWORD ?
.code
    mov eax,6
    .IF eax > val2
        mov result,1
    .ENDIF
```



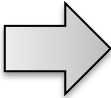
```
    mov eax,6
    cmp eax,val2
    jle @C0001
    mov result,1
@C0001:
```

Conditional Control Flow Directives

- 2. Signed and Unsigned Comparisons

- Comparing Registers

- The assembler cannot determine whether the values are signed or unsigned
 - The assembler defaults to an unsigned comparison (note the use of the JBE instruction)

<pre>mov eax,6 mov ebx,val2 .if eax > ebx mov result,1 .endif</pre>		<pre>mov eax,6 mov ebx,val2 cmp eax, ebx jbe @C0001 mov result,1 @C0001:</pre>
--	---	---

- 3. Compound Expressions

- Using the .IF directive with logical OR and AND operators

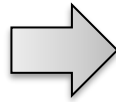
```
.IF expression1 || expression2
    statements
.endif
```

```
.IF expression1 && expression2
    statements
.endif
```

Conditional Control Flow Directives

- 3. Compound Expressions
 - SetCursorPosition Example
 - The SetCursorPosition procedure performs range checking on its two input parameters, DH and DL
 - The Y-coordinate (DH) must be between 0 and 24
 - The X-coordinate (DL) must be between 0 and 79
 - If either is found to be out of range, an error message is displayed

```
SetCursorPosition PROC
.data
BadXCoordMsg BYTE "X-Coordinate out of range!",0Dh,0Ah,0
BadYCoordMsg BYTE "Y-Coordinate out of range!",0Dh,0Ah,0
.code
    .IF (dl < 0) || (dl > 79)
        mov     edx,OFFSET BadXCoordMsg
        call    WriteString
        jmp     quit
    .ENDIF
    .IF (dh < 0) || (dh > 24)
        mov     edx,OFFSET BadYCoordMsg
        call    WriteString
        jmp     quit
    .ENDIF
    call Gotoxy
quit:
    ret
SetCursorPosition ENDP
```



```
.code
; .IF (dl < 0) || (dl > 79)
    cmp     dl, 000h
    jb      @C0002
    cmp     dl, 04Fh
    jbe     @C0001
@C0002:
    mov     edx,OFFSET BadXCoordMsg
    call    WriteString
    jmp     quit
; .ENDIF

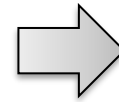
@C0001:
; .IF (dh < 0) || (dh > 24)
    cmp     dh, 000h
    jb      @C0005
    cmp     dh, 018h
    jbe     @C0004
@C0005:
    mov     edx,OFFSET BadYCoordMsg
    call    WriteString
    jmp     quit
; .ENDIF

@C0004:
    call    Gotoxy
quit:
    ret
```

Conditional Control Flow Directives

- 3. Compound Expressions
 - College Registration Example
 - Suppose a college student wants to register for courses
 - We will use two criteria to determine whether or not the student can register
 - The first is the person's grade average, based on a 0 to 400 scale, where 400 is the highest possible grade
 - The second is the number of credits the person wants to take
 - A multiway branch structure can be used, involving .IF, .ELSEIF, and .ENDIF

```
.data
TRUE = 1
FALSE = 0
gradeAverage WORD 275      ; test value
credits       WORD 12       ; test value
OkToRegister  BYTE ?
.code
    mov OkToRegister,FALSE
    .IF gradeAverage > 350
        mov OkToRegister,TRUE
    .ELSEIF (gradeAverage > 250) && (credits <= 16)
        mov OkToRegister,TRUE
    .ELSEIF (credits <= 12)
        mov OkToRegister,TRUE
    .ENDIF
```



```
    mov byte ptr OkToRegister,FALSE
    cmp word ptr gradeAverage,350
    jbe @C0006
    mov byte ptr OkToRegister,TRUE
    jmp @C0008
@C0006:
    cmp word ptr gradeAverage,250
    jbe @C0009
    cmp word ptr credits,16
    ja  @C0009
    mov byte ptr OkToRegister,TRUE
    jmp @C0008
@C0009:
    cmp word ptr credits,12
    ja  @C0008
    mov byte ptr OkToRegister,TRUE
@C0008:
```

Conditional Control Flow Directives

- 4. Creating Loops with .REPEAT and .WHILE
 - The .REPEAT and .WHILE directives
 - They offer alternatives to writing your own loops with CMP and conditional jump instructions
 - The .REPEAT directive executes the loop body before testing the runtime condition following the .UNTIL directive
 - The .WHILE directive tests the condition before executing the loop

```
.REPEAT
    statements
.UNTIL condition
```

```
.WHILE condition
    statements
.ENDW
```

- E.g.) displaying the values 1 through 10

```
mov eax,0
.WHILE eax < 10
    inc  eax
    call WriteDec
    call Crlf
.ENDW
```

```
mov eax,0
.REPEAT
    inc  eax
    call WriteDec
    call Crlf
.UNTIL eax == 10
```

Conditional Control Flow Directives

- 4. Creating Loops with .REPEAT and .WHILE
 - Example: Loop Containing an IF Statement
 - A pseudocode for an IF statement nested inside a WHILE loop:
- An assembly language version

```
while( op1 < op2 )  
{  
    op1++;  
    if( op1 == op3 )  
        X = 2;  
    else  
        X = 3;  
}
```

```
.data  
X    DWORD 0  
op1  DWORD 2  
op2  DWORD 4  
op3  DWORD 5  
.code  
    mov eax,op1  
    mov ebx,op2  
    mov ecx,op3  
    .WHILE eax < ebx  
        inc eax  
        .IF eax == ecx  
            mov X,2  
        .ELSE  
            mov X,3  
        .ENDIF  
    .ENDW
```